z0p: A Zero-Dependency Transparent STARK Stack in Safe Rust

Technical report, specification overview, and measured evaluation

nul0
Future Proofs Tech
futureproofs@proton.me

November 2026

Abstract

We present **z0p**, a complete zero-knowledge proving stack implemented from first principles in safe Rust with no dependencies beyond the compiler and the operating system. Binary fields, the BabyBear prime field, a quartic extension field, BLAKE3-256, Merkle trees, a parallel NTT, FRI, sumcheck, univariate and multilinear polynomial commitments, LogUp lookups, four STARK-style AIR systems, hash-folded IVC, a minimal register-machine AIR, and zero-knowledge masking are owned by a single crate under `#![forbid(unsafe_code)]`. The construction is transparent: no trusted setup, no elliptic curves, no pairings. Security reduces to BLAKE3-256, Reed–Solomon proximity testing, and Fiat–Shamir.

Production parameters target a *conjectured* 100-bit query/proof-of-work schedule: rate 1/16, 40 queries, and 20-bit grinding. Algebraic challenges are drawn from the quartic extension $\mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - u)$, $\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 11)$, giving approximately 124 bits of per-challenge algebraic entropy. On a 16-core reference machine, release profile, `rustc` 1.99.0, a 1,048,576-step Fibonacci AIR proves in 1.73355 s (1.653 μs/step, 604,872 steps/s), verifies in 1.13 ms, and produces a 203,553-byte (198.8 KiB) proof. Verification is essentially independent of trace size: FRI domain points are derived on demand rather than materialized.

The artifact is validated by 277 library tests, 13 security tests with 479 hostile attempts, 10 red-team tests with 37 targeted attacks plus a 500-input garbage storm, 11 hardening tests including exhaustive per-bit AIR decoder fuzzing and 14,000 random mutations across seven encodings, and 5 documentation tests. Every measured forgery is rejected; hostile input fails closed with `Err`, never a panic. The system is explicitly *unaudited*: the contribution is an auditable engineering artifact and a reproducible evidence package, not a claim of production-grade end-to-end soundness.

Contents

1 Introduction **3**
1.1 Related work . . . 3

2 Design goals and threat model **3**

3 Architecture **4**

4 Preliminaries **5**

5 FRI polynomial commitment **5**
5.1 Parameters and commitment . . . 5
5.2 Arity-4 folding . . . 6
5.3 Proof and verification . . . 6

5.4 Conjectured soundness . . . 7

6 Sumcheck and multilinear polynomials 7

7 Univariate, Gemini, and BaseFold commitments 7
7.1 UPCs . . . 7
7.2 Gemini . . . 7
7.3 BaseFold prototype . . . 8

8 Lookup arguments 8
8.1 LogUp in clear . . . 8
8.2 Committed LogUp . . . 8

9 AIR systems 8
9.1 Fibonacci AIR . . . 8
9.2 Range check AIR . . . 9
9.3 Table membership AIR . . . 9
9.4 Hash-folded IVC . . . 9
9.5 Register-machine execution . . . 9
9.6 Recursion fragment . . . 9

10 Zero-knowledge masking 10

11 Wire format and parameters 10

12 Security analysis 11

13 Implementation and performance 11

14 Evaluation and evidence 12

15 Limitations and future work 12

16 Conclusion 13

17 Detailed protocol layouts 13
17.1 Transcript and proof-body conventions . . . 13
17.2 FRI query layout . . . 13
17.3 AIR opening layout . . . 14

18 Formal security statements 14

19 Implementation and audit map 15

A Parameter quick reference 15

B API map 16

C Glossary 16

D Full benchmark table 17

E Per-module test counts 17

F Reproducibility checklist 18

1 Introduction

Practical zero-knowledge infrastructure asks its users to trust a large surface: pairing-friendly curves whose security assumptions are contested, multi-party setup ceremonies, deep dependency trees, and protocol behavior described in papers but not reproducible from the code that runs. At the same time, large-scale quantum computing threatens every deployed pairing- and discrete-log-based proof system. The systems that survive that transition will be hash-based, transparently set up, small enough to audit, and tested hard enough to believe.

Contributions. z0p is, to our knowledge, one of the smallest complete STARK stacks—fields, hash, Merkle trees, NTT, polynomial commitments, AIR systems, lookups, and zero-knowledge masking—implemented from scratch in safe Rust with zero runtime dependencies:

- A transparent, plausibly post-quantum design: BLAKE3-256 digests and FRI Reed–Solomon commitments over BabyBear. No curves, no ceremonies.
- Production parameter sizing in one call (`Params::for_security(..., 100)`): blowup 16, queries, grinding, final layer, and cap height jointly optimized for a conjectured 100-bit query/PoW schedule. All algebraic challenges use a quartic extension field.
- Competitive scalar performance with parallelism over `std::thread` only: 1M-step proofs at approximately 605k steps/s, millisecond verification, and logarithmically growing proofs.
- Adversarial validation as a first-class artifact: every forgery class we could construct is tested continuously, including a fuzzer that once caught a genuine proof-malleability hole (since closed by binding dimensions into the transcript).

Status. Version 0.2.1 is unaudited research software. The approximately 100-bit figure is a query/proof-of-work schedule under a FRI heuristic, not an end-to-end soundness theorem. The paper states exactly what is enforced by code, what is conjectured, and what remains out of scope.

1.1 Related work

STARKs [2] and the FRI proximity test [3], strengthened by DEEP-FRI [4] and compiled via BCS-style Fiat–Shamir [1], are our direct lineage. Production STARK toolchains (notably Plonky2/Polygon Zero [8] and RISC Zero) share the BabyBear-plus-FRI engineering core but retain larger dependency footprints. We differ in taking dependency-freedom and auditable minimalism as hard constraints, and in publishing adversarial test evidence alongside performance claims. Our lookups follow the LogUp logarithmic-derivative design [7]. The hash is BLAKE3 [6]; Fiat–Shamir is classical [5].

2 Design goals and threat model

Goals. (1) From zero: every cryptographic layer is implemented in the crate; `cargo tree` is one node. (2) Transparent and plausibly post-quantum: BLAKE3 and Reed–Solomon proximity testing; no curves, pairings, or setup. (3) Safe by construction: `#![forbid(unsafe_code)]`, fallible public APIs, canonical fail-closed decoding, hostile parameters re-validated at every entry point. (4) Quantified security: explicit query and proof-of-work budgets. (5) Evidence over claims: adversarial suites, byte-exact proof-size assertions, and a pinned benchmark regression gate.

Non-goals. z0p is not an audited production system, a general circuit compiler, a zkVM, a private-table lookup protocol, or a side-channel-hardened hardware token. The `lookup` and `sumcheck` APIs expose tables in clear; `lookup_pcs` hides columns from the wire but is not zero-knowledge; `basefold`, `gemini`, `lookup_pcs`, `vm`, and `recursion` are prototypes.

Threat model. The verifier is the adversary-facing surface. Untrusted inputs are proof and commitment bytes, public parameters (including hand-built `Params` structs whose fields are public), and public statements. The verifier must fail closed with `Err`, never panic, and never perform unbounded allocation or work. It must derive all challenge-dependent data from the transcript and bind every public input before sampling challenges.

Trust base. The Rust compiler and standard library; BLAKE3-256 as a collision-resistant hash; the Reed–Solomon/FRI proximity heuristic and Fiat–Shamir in the (quantum) random-oracle model; the correctness of the field and NTT implementations, cross-tested against independent Horner/`%` references. No trusted setup, elliptic-curve group, pairing, or discrete-log assumption is used.

3 Architecture

Witness / trace → interpolate and coset LDE (parallel NTT, BabyBear) → composition polynomial (AIR constraints divided by vanishing polynomials, randomized by Fiat–Shamir alphas) → FRI commit (arity-4 folds, Merkle caps, PoW) → proof (caps, final polynomial, query openings, trace openings) → verifier (replay transcript, check paths, folds, degree, identities) → accept/reject.

Layer	Modules	Responsibility
Algebra	`gf2`, `babybear`, `ext4`	Word fields, NTT-friendly prime field, quartic challenge field
Hashing	`hash`, `merkle`	BLAKE3-256, domain separation, caps, index-bound paths
Fiat–Shamir	`transcript`, `rng`	Streaming transcript, challenges, grinding, ChaCha20 CSPRNG
Polynomial core	`ntt`, `mle`, `sumcheck`, `fri`	Coset NTT, multilinear tables, sumcheck, FRI
Commitments	`upcs`, `gemini`, `basefold`	Univariate and multilinear opening prototypes
Lookups	`lookup`, `lookup_pcs`	LogUp in clear and committed-input LogUp
Statements	`air`, `range`, `table`, `fold`, `vm`, `recursion`	STARKs, IVC, execution, recursion fragment
Privacy	`blind`	Vanishing-multiple masking
Wire format	`codec`	Canonical versioned encoding, fail-closed decoding
Parallelism	`par`	Deterministic `std::thread` data parallelism

Table 1: Layer ownership in the crate.

Invariants. The implementation enforces a bilateral rate gate $d + 1 \leq M/2$; transcript binding of dimensions, commitments, public inputs, and PoW nonces; authenticated next rows; committed leaves with index-bound Merkle paths; fail-closed canonical decoding; a non-vacuous FRI final-degree check; and no `unsafe` or runtime dependencies.

4 Preliminaries

Fields. Program words use binary fields $GF(2^8)$, $GF(2^{64})$, $GF(2^{128})$. Polynomial domains use BabyBear

$$p = 2^{31} - 2^{27} + 1 = 2013265921, \qquad p - 1 = 2^{27} \cdot 15,$$

with generator 31. Multiplicative subgroups of order 2^k exist for $k \leq 27$. Elements serialize as four canonical little-endian bytes; multiplication uses Barrett reduction with $\mu = \lfloor 2^{64}/p \rfloor$; inversion is Fermat ($a^{p-2}$, mapping zero to zero); batch inversion uses Montgomery's trick.

All algebraic challenges use the tower

$$\mathbb{F}_{p^2} = \mathbb{F}_p[u]/(u^2 - 11), \qquad \mathbb{F}_{p^4} = \mathbb{F}_{p^2}[v]/(v^2 - u),$$

represented as $a + bu + cv + duv$ with four canonical coordinates. 11 is a quadratic non-residue in $\mathbb{F}_p$, and u is a non-square in $\mathbb{F}_{p^2}$. Inversion uses the tower norm: for $x = c + dv$, $x^{-1} = (c - dv)/(c^2 - ud^2)$, so one $\mathbb{F}_{p^2}$ inversion suffices.

Domains and NTT. The trace domain is $H = \{1, \omega, \ldots, \omega^{N-1}\}$; the evaluation domain is the coset $D = \gamma \cdot \langle \Omega \rangle$ of size $M = N\beta$, with coset shift $\gamma = 31$ and $M \leq 2^{27}$. The vanishing polynomial is $Z_H(x) = x^N - 1$. The NTT is a table-driven Cooley–Tukey transform with bit-reversal, deterministic parallel stages, and the inverse scaling folded into the last stage. Reed–Solomon extension interpolates and evaluates on the coset.

BLAKE3 and Merkle trees. Unkeyed BLAKE3-256 is implemented from the specification: 7-round ARX compression, the standard message permutation, flags `CHUNK_START`, `CHUNK_END`, `PARENT`, `ROOT`, 1024-byte chunks, and Bao-style merge-while-even stacking. Column trees use

$$\text{leaf}(d) = H(0x00 \| d), \qquad \text{node}(l, r) = H(0x01 \| l \| r).$$

FRI layers commit one 64-byte leaf per quad (four 16-byte $\mathbb{F}_{p^4}$ values); AIR trace layers commit one 64-byte leaf per eight joint rows. Caps expose an upper tree layer of 2^h digests, and every path direction bit is bound to the claimed leaf index. Cap height is capped at 8.

Transcript. The transcript is append-only and length-prefix framed. Each absorb writes (label_len, label, data_len, data) into a streaming BLAKE3 state. A challenge clones that state, appends (label_len, label, counter), finalizes, and feeds the output back. This is byte-identical to hashing the full concatenation but independent of history length. $\mathbb{F}_p$ challenges reduce a 64-bit word modulo p; query indices mask low bits for power-of-two domains; $\mathbb{F}_{p^4}$ challenges reduce four coordinates.

Proof of work. `grind(label,d)` finds the smallest nonce with at least d leading zero bits under the history commitment, using a banded parallel search identical to a sequential scan. Protocol parameters cap grinding at 32 bits; `grind` fails closed above 32. `verify_grind` checks the nonce and absorbs it only on success. Queries are sampled after the nonce is absorbed.

5 FRI polynomial commitment

5.1 Parameters and commitment

A FRI parameter set is

$$(\beta, Q, F, g, h), \qquad \beta \geq 2, \quad Q \geq 1, \quad 2 \leq F \leq 1024, \quad g \leq 32, \quad h \leq 8,$$

with $F < M$ (at least one fold) and M/F a power of four. `commit` maps n coefficients to $M = n\beta$ coset evaluations with a shifted NTT. `commit_evals` accepts already-expanded LDE values with a declared degree bound $d < M$. Both paths validate parameters before the encoding NTT, reject empty or non-power-of-two lengths, enforce $M \leq 2^{27}$, and bind (M, n, d) under `fri-dims` before any challenge. The rate gate

$$d + 1 \leq M/2$$

is enforced at prove and verify.

5.2 Arity-4 folding

Layers are bit-reversed. Each adjacent quad sits at $(x, -x, j_0 x, -j_0 x)$ for the fixed fourth root j_0, sharing x^4. Writing $P(X) = A + BX + CX^2 + DX^3$ in $Y = X^4$, the fold is

$$\text{fold}(e_0, e_1, e_2, e_3) = A + r_0 B + r_1 C + r_2 D,$$

with

$$\begin{aligned} A &= (e_0 + e_1 + e_2 + e_3)/4, \\ B &= ((e_0 - e_1) - j_0(e_2 - e_3))/(4x), \\ C &= ((e_0 + e_1) - (e_2 + e_3))/(4x^2), \\ D &= ((e_0 - e_1) + j_0(e_2 - e_3))/(4x^3), \end{aligned}$$

and $(r_0, r_1, r_2) \in \mathbb{F}_{p^4}^3$. Point inverses are base-field values and are batch-inverted once per layer. Each fold quarters the degree.

5.3 Proof and verification

A proof carries the final polynomial in the clear, one opening per query (four $\mathbb{F}_{p^4}$ values per folded quad plus the middle Merkle path segment), and a PoW nonce. The verifier:

1. validates dimensions, cap shapes, fold count, and the rate gate;
2. absorbs dimensions, cap digests, and the final polynomial;
3. checks the PoW nonce against the replayed history;
4. verifies the final layer by divided differences against residual $d \gg 2f$ at the true folded domain points;
5. recomputes the final layer cap from `final_poly` and compares it to the commitment;
6. samples query indices after grinding and checks every Merkle path and fold relation, chaining intermediate parents forward and the last parent into `final_poly`.

The final degree check is non-vacuous: a one-point final layer would let any value interpolate a degree-0 polynomial, so $F \geq 2$ is mandatory and

$$(d \gg 2f) + 1 < F$$

is enforced. A fold-free instance ($F = M$) is rejected because it has no query openings binding `final_poly` to the commitment.

Shape	Rate ρ	Queries Q	Grinding	Conjectured query+PoW
Bench	1/8	12	0	≈ 18 bits
Production	1/16	40	20	≈ 100 bits $(80 + 20)$

Table 2: Conjectured FRI query/PoW schedule. Algebraic error is separate.

5.4 Conjectured soundness

Under the Johnson-radius heuristic, a word at distance δ from the code survives one query with probability at most $1 - \delta$, where $\delta \approx 1 - \sqrt{\rho}$.

These counts exclude finite-field algebraic error. FRI folds, STARK compositions, UPCs/Gemini points and betas, sumcheck rounds, and lookup identities use $\mathbb{F}_{p^4}$, so a degree-d identity collides with probability about d/p^4 per challenge ($\approx$ 124 bits). This remains a query/PoW formula, not an end-to-end theorem.

6 Sumcheck and multilinear polynomials

A multilinear polynomial with n variables stores 2^n evaluations in MSB-first order. `evaluate` folds the least-significant variable first. The equality polynomial is

$$\mathrm{eq}(r, x) = \prod_i (r_i x_i + (1 - r_i)(1 - x_i)).$$

A summand is a sum of terms $f(x) = \sum_k c_k \prod_{j \in \mathrm{factors}_k} \mathrm{mle}_j(x)$ with $c_k \in \mathbb{F}_{p^4}$. Sumcheck proves $S = \sum_{x \in \{0,1\}^n} f(x)$ by sending round polynomials of degree at most the maximum factor count. Verification checks $g_i(0) + g_i(1) = g_{i-1}(r_{i-1})$, samples $r_i \in \mathbb{F}_{p^4}$, and finally checks the oracle value. If the oracle is committed, each final value is opened at the challenge point.

7 Univariate, Gemini, and BaseFold commitments

7.1 UPCs

`upcs` commits a univariate polynomial as a Merkle-committed Reed–Solomon LDE. To prove $g(\beta) = y$, the prover commits $q = (g - y)/(X - \beta)$ and proves both g and q low-degree with FRI; the verifier checks $(X - \beta)q = g - y$ at the quotient's query points. Multi-point openings use I interpolating the claims and $Z = \prod_j (X - p_j)$, with $q = (g - I)/Z$ and $g = I + Zq$. When the points determine the polynomial ($k =$ poly_len), the quotient is zero and the verifier checks g against the interpolant directly.

7.2 Gemini

Gemini commits the monomial-coefficient encoding $g_0(X) = \sum_b c_b X^{\mathrm{index}(b)}$ and proves $f(r) = y$ by folding one variable at a time:

$$g_{i-1}(X) = A_i(X^2) + XB_i(X^2), \qquad g_i(Y) = A_i(Y) + r_i B_i(Y).$$

Each committed layer is opened once with a multi-point UPCs proof at all points its relations use. Layer commitments are absorbed before the fold betas. Proof size is linear, not constant. Each fold identity has degree below 2^{n-i+1}, so one random check misses a nonzero identity with probability at most $2^{n-i+1}/p^4$.

7.3 BaseFold prototype

BaseFold commits the coefficient encoding with FRI and reduces $f(r) = y$ to a coefficient-oracle evaluation $c(\rho)$ with sumcheck. The prover folds the LDE with the Möbius/eq rule

$$(1 - \rho_i)E(X) + \rho_i O(X)$$

rather than the coefficient-encoding rule $E(X) + \rho_i O(X)$, so the final FRI constant is exactly $c(\rho)$. The verifier recomputes

$$w(\rho) = \prod_i (1 - \rho_i + \rho_i r_i)$$

and checks $f(r) = c(\rho)w(\rho)$. Proofs carry one root, $n - 1$ layer roots, sumcheck rounds, and one pair plus one Merkle path per layer per query. Optional caps store a digest frontier per layer and hash it back to the layer root. Batched multi-point openings use the multilinear basis table and are capped at 1024 points. The prototype is not wired into the AIR/STARK stack and has no end-to-end soundness proof; it backs the committed lookup.

8 Lookup arguments

8.1 LogUp in clear

For witness w and table t padded to powers of two, the prover computes first-index multiplicities m, samples β avoiding poles, and forms

$$q_i^w = \frac{1}{\beta + w_i}, \qquad q_j^t = \frac{m_j}{\beta + t_j}.$$

Four sumchecks establish the rational identity $\sum_i q_i^w = \sum_j q_j^t$ and the read-checks $q_i^w(\beta + w_i) = 1$, $q_j^t(\beta + t_j) = m_j$. The verifier has witness and table in clear and recomputes canonical multiplicities exactly; fractional multiplicities are rejected. The API is sound for membership but not hiding.

8.2 Committed LogUp

`lookup_pcs` commits witness, table, multiplicities, and both quotient columns with BaseFold. The verifier sees five commitments and eight openings. The read-checks use the committed columns directly:

$$\mathrm{eq} \cdot \big(q \cdot (\beta + v) - \mathrm{target}\big) = 0,$$

expanded as $\mathrm{eq}qv + \beta\mathrm{eq}q - \mathrm{eq}\,\mathrm{target}$. A fresh γ folds the total multiplicity weight into the sum-t claim:

$$\sum (q^t + \gamma m) = \sum q^w + \gamma W.$$

The verifier rejects $\gamma = 0$. Every final sumcheck value is opened against its commitment. At 1,024-in-1,024 with $B16Q40G20C4$, the prototype measures 273.01 ms prove, 5.77 ms verify, and 642,443 bytes. It is committed-input but not zero-knowledge and does not constrain columns to the base field.

9 AIR systems

9.1 Fibonacci AIR

Trace $A[0] = a_0$, $B[0] = b_0$, $A[i+1] = B[i]$, $B[i+1] = A[i] + B[i]$, claim $B[N-1] = b_{\text{last}}$. With $w_{\text{last}} = \omega^{N-1}$ and $Z_{\text{trans}}(x) = (x^N - 1)/(x - w_{\text{last}})$:

$$C(x) = \alpha_0 \frac{a' - b}{Z_{\text{trans}}(x)} + \alpha_1 \frac{b' - (a+b)}{Z_{\text{trans}}(x)} + \alpha_2 \frac{a - a_0}{x - 1} + \alpha_3 \frac{b - b_0}{x - 1} + \alpha_4 \frac{b - b_{\text{last}}}{x - w_{\text{last}}}.$$

Five `fib/alpha` challenges; $\deg C < N$ iff the trace is valid. Dividing, not multiplying, by Z_{trans} is load-bearing. Blowup is at least 4. Next-row values are the same committed columns at the ω-shifted LDE index, authenticated by a second Merkle path.

9.2 Range check AIR

Value column V plus n bit columns ($1 \le n \le 24$), with challenges α_{decomp}, $\{\alpha_j\}$, α_{bound}:

$$C = \alpha_{\text{decomp}} \frac{V - \sum_j 2^j C_j}{Z_H} + \sum_j \alpha_j \frac{C_j(C_j - 1)}{Z_H} + \alpha_{\text{bound}} \frac{V - \text{value}}{x - 1}.$$

Booleanity doubles the masked degree; masked callers should size blowup and queries for a rate near $2/\beta$. The verifier caps the declared bound at the largest honest masked degree.

9.3 Table membership AIR

$$C(x) = \alpha_{\text{member}} \frac{\prod_{t \in T}(v - t)}{Z_H(x)} + \alpha_{\text{bound}} \frac{v - \text{value}}{x - 1}.$$

The honest composition bound is $|T|(n + k - 1) - n$ for n rows and k maskers. The tiny-table `for_security` tuner floors blowup at 4 so FRI always has a valid final layer.

9.4 Hash-folded IVC

Segments of `seg_steps` Fibonacci rows are committed independently with explicit in/out claims. The verifier checks chain links, then verifies one shared FRI proof over

$$V = \sum_s \gamma_s C_s$$

with per-step challenges. A far-from-code segment keeps the combination far with overwhelming probability over the gammas. The statement is existential over the number of segments; callers requiring an exact step count must check `proof.steps.len()`. The accumulator retains each segment's LDE, Merkle tree, and composition until finalization.

9.5 Register-machine execution

A public two-register straight-line program runs from a private initial state to a public output. Opcodes are `Add`, `Mul`, `Swap`, `SetA`, `SetB`, and `Halt`; the last row is the public output and has no outgoing transition. Public one-hot selectors and immediate polynomials drive transition numerators T, U:

$$C(x) = \alpha_0 \frac{T(x)}{Z_{\text{trans}}(x)} + \alpha_1 \frac{U(x)}{Z_{\text{trans}}(x)} + \alpha_2 \frac{A(x) - a_{\text{last}}}{x - w_{\text{last}}} + \alpha_3 \frac{B(x) - b_{\text{last}}}{x - w_{\text{last}}}.$$

Plain composition degree is below $2n - 1$, so blowup is at least 8. Masked columns have degree padded $- 1$; the `Mul` transition is quadratic, so the prover declares $2(n + n_{\text{rand}} - 1)$ and the verifier accepts up to $2 \cdot \text{padded} - 2$. Each query reveals the current and shifted rows, so the mask budget is $2Q$. The program and output remain public; there is no control flow or memory.

9.6 Recursion fragment

`recursion` proves one FRI query's fold chain as a small STARK. Public inputs are the outer `lde_len`, leaf, per-depth challenges, the layer-0 quad, and the output; the witness is the sequence of quads and folded outputs. Constraints cover the fold identity, the selected next-layer element, the initial quad, and the final output. The fragment does not yet verify BLAKE3 paths, the final degree check, or transcript derivation inside the AIR.

10 Zero-knowledge masking

Masked proving maps a column T of degree $< n$ to

$$M(x) = T(x) + Z_H(x)R(x),$$

where R has n_{rand} uniform coefficients. Z_H vanishes on every base row, so trace values and AIR quotient identities are preserved. Off-domain, the mask term is nonzero. Evaluations of R at k distinct points are jointly uniform when $n_{\text{rand}} \geq k$ (Vandermonde), giving perfect hiding of query responses.

Protocol	Revealed values per column per query	Minimum n_{rand}
`range`, `table`	one point	Q
`vm`	current and shifted rows	$2Q$
`air`	eight-row leaf and shifted leaf	$16Q$

Table 3: Masking budgets for the values actually revealed.

The composition degree grows accordingly; callers must size `final_size` for the masked LDE. Verification is unchanged, and masked false statements still fail. This is query-response hiding, not full program privacy: public inputs, Merkle roots, and composition commitments remain visible. `lookup`, `sumcheck`, `upcs`, `gemini`, and `basefold` are not hiding.

11 Wire format and parameters

All proofs use a canonical versioned encoding, `VERSION=5`: little-endian integers, $u32$ counts, canonical base and extension fields, 32-byte digests, and 0/1 direction bytes. The decoder rejects trailing bytes, truncation, non-canonical fields, and oversized counts before allocation.

Limit	Value
`MAX_QUERIES`	$2^{20} = 1{,}048{,}576$
`MAX_FOLDS`	64
`MAX_PATH_STEPS`	128
`MAX_CAP_DIGESTS`	256
`MAX_ELEMS`	$2^{27} = 134{,}217{,}728$
`MAX_STEPS` (IVC)	$2^{20} = 1{,}048{,}576$
FRI `MAX_FINAL_SIZE`	1024
maximum Merkle cap height	8
BaseFold `MAX_BATCH`	1024
BaseFold `MAX_VARS`	16
Gemini `MAX_VARS`	20
UPCs `MAX_POINTS`	16
VM `MAX_PROGRAM`	2^{20}
recursion `MAX_FOLDS`	16

Table 4: Hard decoder and protocol limits.

Parameter validation requires power-of-two domains, blowup at least 4 for AIR/fold, at least 8 for Gemini/VM, queries at least 1, final size at least 2 and at most 1024, M/F a power of four, cap height at most 8, grinding at most 32, the rate gate, a non-vacuous residual bound, and an outer recursion `lde_len` at most 2^{27}.

12 Security analysis

Assumptions. BLAKE3 collision/preimage resistance; FRI soundness under the Johnson-radius heuristic; Fiat–Shamir in the (quantum) random-oracle model; standard Reed–Solomon list-decoding bounds. No discrete-log, pairing, or setup assumptions: the construction is Shor-inert by design.

Enforced invariants. Bilateral rate gate; verifier-derived domains and query indices; non-vacuous final divided-difference check; cap/path shape pins before hashing; direction bits bound to claimed indices; `fri-dims` dimension binding; canonical-only decoding; PoW bound to history with queries sampled after; committed read-checks in `lookup_pcs`; correct masked composition bounds in VM; and next-row openings authenticated against the same trace cap.

Closed findings. Internal adversarial review has closed, among others: Merkle path index substitution across all verifiers and hand-rolled FRI/BaseFold walks; one-point FRI final layers and fold-free FRI; final cap not bound to `final_poly`; `lookup_pcs` read-checks using uncommitted shifted MLEs; VM masked `Mul` degree under-declaration; AIR/VM ZK mask budgets undercounting revealed values; batched BaseFold openings using the equality table instead of the multilinear basis table; and prover allocation ordering, cap-height/roundtrip limits, recursion domain bounds, checked buffer arithmetic, and transcript/grinding hardening. Regression tests and reproducer descriptions are in `docs/security-tests.md`.

Residual risk. The system is unaudited. The query/PoW figures are conjectured. The small-table AIR scales with table size. `lookup_pcs` is not zero-knowledge and does not constrain columns to the base field. BaseFold, Gemini, VM, and recursion are prototypes. Side channels in the host OS, compiler, or CPU are out of scope. Do not deploy for high-value statements before an external audit.

13 Implementation and performance

Engineering. Safe Rust, zero dependencies, parallelism over `std::thread` only. NTT stages at least 2^{16}, Merkle levels, FRI folding, composition, and PoW search are parallelized; all routines are bit-identical to sequential references. Key optimizations: Barrett field arithmetic, table-driven NTT, single-chunk hash fast paths, batched composition with geometric vanishing and Montgomery denominators, verifier on-demand domain points, Merkle caps, and a streaming transcript history.

Method. `benches/speed_size.rs`, release profile, median of runs after one warm-up, 16 cores / 31 GB, `rustc` 1.99.0, measured 2026-11-05. Every benchmark verifies its proof once outside the timed region; byte counters assert `to_bytes().len()` exactly. Proof sizes are deterministic. The regression gate allows 3× timing drift and requires byte-exact sizes.

Workload (production query/PoW)	Prove	Verify	Size
AIR 1,024 steps	37.05 ms	0.48 ms	68,573 B
AIR 8,192 steps	49.81 ms	0.59 ms	101,233 B
AIR 65,536 steps	115.49 ms	0.90 ms	141,641 B
AIR **1,048,576** steps	**1,733.55 ms**	**1.13 ms**	**203,553 B**
FRI poly-1,024	21.50 ms	0.31 ms	43,174 B

Table 5: Production AIR/FRI measurements, 2026-11-05.

The 1M-step row is 1.653 μs/step and 604,872 steps/s. Proof size grows logarithmically; verification stays in the low-millisecond range from 1,024 to 1,048,576 steps. On the optional nightly SIMD hash path, a representative run measured 1,693.58 ms for the same row (approximately 2.3% faster); the default scalar path remains the supported stable build.

Construction	Prove	Verify	Size
BaseFold-style, 1,024 coefficients	22.65 ms	0.88 ms	127,658 B
BaseFold-style, cap height 4	22.91 ms	0.63 ms	80,018 B
Gemini v1, 1,024 coefficients	317.96 ms	7.36 ms	887,872 B
`lookup_pcs`, 1,024-in-1,024, C4	273.01 ms	5.77 ms	642,443 B
`vm`, 64 steps, $B16Q12C2$	0.69 ms	0.18 ms	14,933 B

Table 6: Multilinear PCS, committed lookup, and execution measurements.

Cap height 4 reduces the BaseFold proof by 37.3%. Against Gemini, uncapped BaseFold is approximately 14.0× faster to prove, 8.4× faster to verify, and 7.0× smaller; capped is approximately 13.9×, 11.7×, and 11.1×. The BaseFold and committed-lookup rows are prototypes; the VM row is the plain path.

Primitives. BLAKE3 1 MiB: 1.17 ms (856.5 MB/s). BabyBear multiplication: 6.59 ms/1M (151.8 Mops/s). $GF(2^{64})$ multiplication: 272.84 ms/1M (3.7 Mops/s). NTT 2^{14}: 0.32 ms. 64k-leaf Merkle tree: 3.24 ms.

14 Evaluation and evidence

Suite	Tests	Evidence
Library	277	KATs, Horner/`%` cross-checks, parallel determinism, protocol roundtrips
`security.rs`	13	479 hostile attempts, zero accepted
`redteam.rs`	10	37 targeted + 500 garbage, zero accepted/panics
`hardening.rs`	11	Hostile params, exhaustive per-bit AIR decoder fuzz, 14,000 mutations, zero panics
Doc tests	5	Public examples
Total	**316**	Plus coverage-guided `fuzz/decode`

Table 7: Validation matrix on the current tree.

The adversarial suites cover tampering of every query, fold, value, path, digest, commitment, and trace opening; cross-proof transplants; false statements; IVC surgery; codec limits; transcript abuse; exact rate boundaries; masked soundness; and a 500-input garbage storm. Hostile input must fail closed with `Err`, never panic. The gate `python3 tools/check.py` runs fmt, relative Markdown links, rustdoc, pedantic clippy, debug tests, release tests, and the benchmark regression.

15 Limitations and future work

Bench defaults are fast but weak and must not be mistaken for production soundness. The table AIR is small-table by design. The committed LogUp prototype removes clear witness/table inputs but still uses eight separate BaseFold openings, lacks hiding, and does not constrain columns to the base field. Vectorized hashing and the one-query recursion fragment are complete; FRI-backed univariate point and multi-point openings are in place; Gemini and a compact

BaseFold-style prototype exist. The prototype, committed lookup, and program-execution AIR are experimental and not wired into production AIR/STARK paths. Batched openings, a hiding PCS/sumcheck, self-proving recursion, control flow, memory, instruction privacy, and a general zkVM remain roadmap items. No external audit has yet been performed; the suites, gates, versioned wire format, and pinned baselines are the evidence package.

16 Conclusion

z0p demonstrates that a useful, transparent STARK stack can be implemented from first principles in safe Rust with zero dependencies and a small, auditable trusted base. The current artifact is deliberately positioned as pre-audit research software: the engineering, adversarial evidence, formal specification, and measured evaluation are complete enough for external review, while the soundness figures remain conjectured and the prototype components remain explicitly labeled.

17 Detailed protocol layouts

17.1 Transcript and proof-body conventions

All proof bodies use the canonical codec described above. A proof is `VERSION || body`; nested proofs are length-prefixed. The verifier replays exactly the prover's absorb/squeeze sequence. The following table summarizes the principal bodies; the Rust types are normative where this summary is not exhaustive.

Proof type	Body
FRI	final polynomial, PoW nonce, query count, per-query fold witnesses (four $\mathbb{F}_{p^4}$ values plus path)
AIR, range, table, VM, recursion	column cap, FRI commitment and proof, opening count, per-opening index/values/paths (and 64-byte leaves for AIR)
fold (IVC)	step count, per-step claims/cap/openings, shared FRI commitment and proof
BaseFold	sumcheck sum and rounds, layer roots, caps, PoW nonce, queries
UPCs	two FRI instances (statement and quotient), quotient commitment/proof, opening count, per-opening value/leaf/path
Gemini	layer count and commitments, value count, nested UPCs openings
lookup	beta, multiplicities, quotient columns, sums, four nested sumcheck proofs
lookup_pcs	beta, sum, four nested sumcheck proofs, eight BaseFold openings
sumcheck	claimed sum, round count and round evaluations, final values

17.2 FRI query layout

For layer d, a query opens the four values of the quad containing the current index and a Merkle path from the quad's parent to the layer root or selected cap entry. The verifier recomputes the leaf digest from the four values, checks the path and direction bits, folds the quad with the layer challenge, and requires the parent to be the selected value in the next layer. The last parent must equal the corresponding element of `final_poly`. The final layer cap is recomputed from `final_poly`, so the commitment has no unconstrained digest.

17.3 AIR opening layout

An AIR opening contains the natural-order LDE index, the column values at that index, the same columns at the ω-shifted index, two Merkle paths, and (for the Fibonacci AIR) two 64-byte leaves containing eight joint rows each. The verifier checks the row at `(index & 7) * 8`, authenticates the leaf at `index / 8`, recomputes the shifted index, authenticates the second leaf, and evaluates the composition identity at the on-demand domain point. Range/table/VM/fold/recursion openings use analogous row layouts without the eight-row packing.

18 Formal security statements

Assumptions. We state the following conditionally on the standard FRI heuristic and the random-oracle model; they are not unconditional theorems.

Definition 1 (Rate gate)**.** *A FRI instance with declared degree bound d and LDE length M satisfies the rate gate if $d + 1 \le M/2$.*

Lemma 2 (Non-vacuous final check)**.** *If $F \ge 2$ and $(d \gg 2f) + 1 < F$, the final divided-difference check is not vacuous: there exist F points and values for which a polynomial of degree greater than the residual bound is rejected.*

Sketch. With F distinct points, the interpolation space has dimension F. If the residual bound were at least $F - 1$, every value vector would admit a degree-$\le F - 1$ interpolant. Requiring the bound below $F - 1$ leaves a nonempty set of divided differences that must vanish. The implementation checks exactly those differences. □

Lemma 3 (Merkle index binding)**.** *For a path of length k and claimed index i, verification succeeds only if the direction bits equal the low k bits of i and the recomputed root matches. Consequently a valid opening for leaf j cannot be accepted at an index $i \ne j$ with the same path length.*

Sketch. The verifier iterates over the path and rejects the first direction bit that disagrees with the corresponding index bit. If all bits agree, the recomputed position is i; a different leaf j would require a collision in the hash chain or a different direction sequence. □

Lemma 4 (Committed read-check)**.** *In `lookup_pcs`, the read-check summand uses the committed witness, table, and multiplicity columns as MLE factors. A prover that changes the witness/table after the commitment must either open the commitment at the sumcheck point to a different value or produce a sumcheck final value that does not satisfy the committed polynomial; both are rejected by the opening and final-value checks.*

Lemma 5 (Mask hiding, conditional)**.** *If the number of distinct masked evaluations revealed per column is at most n_{rand}, then the mask evaluations are jointly uniform over the revealed points by Vandermonde surjectivity, so the revealed values are independent of the witness.*

Conjecture 6 (Query/PoW schedule)**.** *Under the Johnson-radius heuristic, rate $1/16$, 40 queries, and 20-bit grinding give approximately 100 bits of query/PoW security; rate $1/8$ with 12 queries gives approximately 18 bits. These figures exclude the finite-field algebraic term and are not an end-to-end theorem.*

Closed findings. The following table records the main adversarial findings and the regression that pins each fix.

Finding	Fix	Regression
Merkle index substitution	direction bits bound to index	`merkle::prove`
One-point FRI final layer	$F \geq 2$ + residual check	`fri::fold_free`
Fold-free FRI	reject $F = M$	`fri::fold_free`
Final cap unconstrained	recompute cap from final poly	FRI verification suite
Lookup read-check MLEs	use committed columns	`lookup_pcs::tamper`
VM masked degree	quadratic bound	`vm::zk_final16`
ZK mask budget	AIR $16Q$, VM $2Q$	`zk_rejects_few_maskers`
BaseFold batch table	multilinear basis table	`basefold::multi_point`

19 Implementation and audit map

Concurrency. NTT stages at least 2^{16}, Merkle levels, FRI folding, composition, and PoW search use `std::thread::scope`. Chunk boundaries depend only on lengths; every output is a pure function of its inputs. Tests pin parallel-vs-sequential identity at 2^{16} and 2^{17} NTT sizes and for Merkle batch hashing.

Memory discipline. The verifier derives FRI domain points on demand and never builds the full layer stack. Counts are capped before allocation. Prover trace/composition buffers use checked multiplication. The FRI final layer is capped at 1024; Merkle caps at 256 digests. The fold accumulator retains per-segment state until finalization and documents its $O(\text{segments} \times \text{LDE})$ prover memory.

Audit map. Reviewers should start with `codec`, `transcript`, `merkle`, and `fri`; then fields (`babybear`, `ext4`, `gf2`); then the polynomial core (`ntt`, `mle`, `sumcheck`); then statements (`air`, `range`, `table`, `fold`, `vm`, `recursion`); then masking and the experimental commitments/lookups. `tests/security.rs`, `tests/redteam.rs`, and `tests/hardening.rs` map each invariant to a regression.

Reproduction. The full gate is

```
python3 tools/check.py
```

which runs formatting, relative Markdown links, rustdoc, pedantic clippy, debug tests, release tests, and the benchmark regression. The baseline pins all 24 rows; proof sizes are byte-exact and times are allowed 3× drift.

A Parameter quick reference

Constructor	Key validation
`fri::Params::new`	blowup ≥ 2, queries ≥ 1, final $2 \ldots 1024$
`air::Params::new`	N power of two ≥ 2, blowup ≥ 4, $M \leq 2^{27}$
`range::Params::new`	bits $1 \ldots 24$, blowup ≥ 2
`table::Params::new`	table checked at prove/verify, blowup ≥ 2
`fold::Params::new`	segment ≥ 2, blowup ≥ 4
`vm::Params::new`	blowup ≥ 8, program $\leq N - 1$

Constructor	Key validation
`recursion::Params::new`	folds $1 \dots 16$, outer LDE $\leq 2^{27}$
`basefold::Params::new`	blowup ≥ 2, cap ≤ 8, vars ≤ 16
`gemini::Params::new`	blowup ≥ 8, vars ≤ 20
`upcs`	points ≤ 16, blowup ≥ 2, LDE $\leq 2^{27}$

B API map

Module	Main entry points
`babybear`	`Field`, `batch_invert`
`ext4`	`Ext4`, `from_base`, `mul_base`, `inv`
`gf2`	`F8`, `F64`, `F128`, `splitmix64`
`hash`	`blake3_256`, `Hasher`
`merkle`	`MerkleTree`, `hash_leaf`, `hash_node`, `capped_shape`
`transcript`	`Transcript`, `challenge_*`, `grind`, `verify_grind`
`ntt`	`forward`, `inverse`, `interpolate`, `evaluate_on_coset`, `evaluate`
`fri`	`Params`, `commit`, `commit_evals`, `open`, `verify`
`mle`	`Mle`, `eq_evals`, `evaluate`, `fix_last`
`sumcheck`	`Term`, `Proof`, `prove`, `verify`, `interpolate_at`
`upcs`	`commit`, `open`, `open_multi`, `verify`, `verify_multi`
`gemini`	`commit`, `commit_evals`, `open`, `verify`
`basefold`	`commit`, `open`, `open_multi`, `verify`, `verify_multi`
`lookup`	`prove`, `verify`
`lookup_pcs`	`prove`, `verify`, `Commitments`
`air`	`Params`, `build_trace`, `prove`, `prove_zk`, `verify`
`range`	`Params`, `build_columns`, `prove`, `prove_zk`, `verify`
`table`	`Params`, `prove`, `prove_zk`, `verify`
`fold`	`Params`, `Accumulator`, `verify`
`vm`	`Params`, `Op`, `prove`, `prove_zk`, `verify`
`recursion`	`Params`, `prove`, `verify`
`blind`	`mask_coeffs`, `masked_column_degree`
`rng`	`ChaCha20`, `os_seed`, `fresh_stream`

C Glossary

Term	Meaning
AIR	Algebraic Intermediate Representation: trace plus transition/boundary constraints
FRI	Fast Reed–Solomon Interactive Oracle Proof of Proximity
LDE	Low-degree extension: evaluations on a larger coset domain
MLE	Multilinear extension/polynomial
PCS	Polynomial commitment scheme
PoW	Proof of work used as a Fiat–Shamir grinding step
STARK	Scalable Transparent Argument of Knowledge
UPCs	Univariate polynomial commitment scheme

Term	Meaning
ZK	Zero knowledge

D Full benchmark table

All 24 rows measured 2026-11-05, release profile, median of runs after one warm-up, 16 cores / 31 GB, `rustc` 1.99.0. Times are hardware-dependent; sizes are deterministic and wire-exact.

Row	Prove	Verify	Size
`blake3 1MiB`	1.17 ms	–	–
`gf2 F64 mul`	272.84 ms	–	–
`bb field mul`	6.59 ms	–	–
`ntt forward` 2^{14}	0.32 ms	–	–
`fri poly256 B4 Q12`	0.15 ms	0.08 ms	11,522 B
`air fib64`	0.17 ms	0.11 ms	15,465 B
`range 16-bit`	0.10 ms	0.07 ms	5,217 B
`table 4-entry`	0.04 ms	0.04 ms	4,673 B
`lookup 64-in-16`	0.19 ms	0.09 ms	2,897 B
`lookup-pcs 1024-in-1024 B16Q40G20C4`	273.01 ms	5.77 ms	642,443 B
`vm 64-step B16Q12C2`	0.69 ms	0.18 ms	14,933 B
`fold 4x8 fib`	0.19 ms	0.19 ms	16,845 B
`basefold P1024 B16Q40G20`	22.65 ms	0.88 ms	127,658 B
`basefold-capped P1024 B16Q40G20C4`	22.91 ms	0.63 ms	80,018 B
`gemini P1024 B16Q40G20`	317.96 ms	7.36 ms	887,872 B
`merkle 64k leaves`	3.24 ms	–	–
`air fib1024`	2.01 ms	0.20 ms	28,257 B
`fri prod B16Q40G20C4F16`	10.59 ms	0.25 ms	29,378 B
`air prod B16Q40G20C4F16`	11.99 ms	0.27 ms	33,061 B
`fri prod P1024 B16Q40G20C4F16`	21.50 ms	0.31 ms	43,174 B
`air prod N1024 B16Q40G20C4F16`	37.05 ms	0.48 ms	68,573 B
`air prod N8192 B16Q40G20C4F32`	49.81 ms	0.59 ms	101,233 B
`air prod N65536 B16Q40G20C4F16`	115.49 ms	0.90 ms	141,641 B
`air prod N1048576 B16Q40G20C4F16`	1,733.55 ms	1.13 ms	203,553 B

E Per-module test counts

Module	Tests	Focus
`air`	23	AIR, ZK, caps, next-row binding, codec
`babybear`	7	Arithmetic, Barrett, inversion, roots
`basefold`	23	Openings, caps, batching, codec
`ext4`	1	Tower laws, inversion, encoding
`blind`	5	Masking degree and randomness
`codec`	2	Counts, canonical fields, trailers
`fold`	15	IVC chains, caps, shifted openings
`fri`	21	FRI roundtrips, rate gates, caps, degree
`gf2`	7	Binary-field arithmetic and inverses
`hash`	6	BLAKE3 KATs and incremental paths

Module	Tests	Focus
`lookup`	8	Membership and multiplicity binding
`lookup_pcs`	6	Committed lookup and tamper
`merkle`	7	Paths, padding, caps, index binding
`mle`	5	Evaluation, fixing, equality
`ntt`	7	Horner cross-checks and parallelism
`par`	2	Parallel determinism and thresholds
`range`	17	Range AIR, ZK, caps, bounds
`upcs`	28	Point/multi-point openings and tamper
`gemini`	17	Fold chains, openings, transcript
`recursion`	12	Fold-chain AIR and tamper
`rng`	7	ChaCha20 vectors and hygiene
`sumcheck`	7	Sums, rounds, transcript
`table`	18	Membership, ZK, caps, bounds
`transcript`	11	FS, PoW, domains, streaming history
`vm`	15	Execution, ZK, final-layer degree, codec

F Reproducibility checklist

1. `cargo fmt -check`
2. `python3 tools/check_links.py`
3. `RUSTDOCFLAGS="-D warnings" cargo doc -no-deps`
4. `cargo clippy -all-targets - -D warnings`
5. `cargo test` and `cargo test -release`
6. `cargo bench` (24 rows)
7. `python3 tools/check.py` (full gate)
8. Optional: `RUSTC_BOOTSTRAP=1 cargo test -features simd`

References

[1] E. Ben-Sasson, A. Chiesa, and N. Spooner. Interactive oracle proofs. In *TCC 2016*.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018.

[3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *ICALP 2018*.

[4] E. Ben-Sasson, D. Bentov, Y. Horesh, and M. Riabzev. DEEP-FRI: Sampling outside the box improves soundness. Cryptology ePrint Archive, Report 2019/336, 2019.

[5] A. Fiat and A. Shamir. How to prove yourself: practical solutions to identification and signature problems. In *CRYPTO 1986*.

[6] J.-P. Aumasson, S. Neves, J. O'Connor, and Z. Wilcox-O'Hearn. BLAKE3: one function, fast everywhere. https://github.com/BLAKE3-team/BLAKE3, 2020.

[7] U. Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022.

[8] Polygon Zero Team. Plonky2: fast recursive arguments with PLONK and FRI. https://github.com/0xPolygonZero/plonky2, 2022.